Apresentação Comentada Do Jogo De Truco

Este documento foi preparado para explicar a estrutura e a lógica do jogo em Pascal de forma simples, mostrando os trechos principais do código com destaque azul.

1. Estruturas Principais Do Jogo

Nesta parte, criamos os registros que representam os elementos do jogo. A carta, que guarda o seu número, o naipe, que guarda os quatro tipos de naipe(paus, copas, espada e ouro) e a força, que indica qual carta se sobressai a outra.

O baralho funciona como pilha e a mão do jogador e do computador funcionam como lista.

```
type
  iCarta = record
    carta: integer;
    naipe: string;
    forca: integer;
  end;
  iBaralho = array[1..40] of iCarta;
```

Trecho do arquivo linhas 05 a 11.

- **iCarta** representa o registro;
- **carta:** variável do número das cartas;
- **naipe:** guarda o naipe das cartas(paus, copas, espadas e ouro);
- **forca:** indica qual carta se sobressai sobre a outra;
- **iBaralho:** array que guarda 40 posições para o registro.

2. Inserindo Baralho no Registro

Aqui há um procedimento que faz a inserção dos números para a variável carta e dos naipes para a variável naipe.

```
procedure inserirBaralho(var baral: iBaralho);
var i, n, x: integer;
    naipes: array[1..4] of string;
begin
  writeln('          BARALHO INICIADO E EMBARALHADO. BORA PRO JOGO!');
  writeln;
  naipes[1]:= 'Paus'; naipes[2]:= 'Copas';
  naipes[3]:= 'Espadas'; naipes[4]:= 'Ouro';
  x:= 1;
  for i:= 1 to 12 do
    for n:= 1 to 4 do
    begin
      if (i < 8) or (i > 9) then
      begin
        baral[x].carta := i;
        baral[x].naipe := naipes[n];
        x := x + 1;
      end;
    end;
end;
```

Trecho do arquivo linhas 13 a 32.

- Identifica os naipes em sua devida variável;
- Utiliza um for de 1 a 12 para cartas, e um for de 1 a 4 para os naipes;
- Utiliza uma condição(if (i < 8) or (i > 9) then) para não inserir as cartas 8 e 9.

3. Embaralhando o Baralho

Este procedimento foi criado para embaralhar as cartas, tirando-as da sequência deixada pelo procedimento de "inserirBaralho". O procedimento utiliza uma lógica incrível. Ele tem uma variável local temporária (temp), para guardar um valor e substituí-lo em outra posição do baralho.

```
procedure embaralhar(var baral: iBaralho);
var i, j: integer; temp: iCarta;
begin
  Randomize;
  for i := 40 downto 1 do
  begin
    j := Random(i) + 1;
    temp := baral[i];
    baral[i] := baral[j];
    baral[j] := temp;
  end;
end;
```

Trecho do arquivo linhas 34 a 45.

- Roda um for de 40 a 1 (downto);
- Variável J sorteia um número de i (40 inicialmente) até 1;
- Variável temp guarda a carta na posição de i;
- É substituída a carta que está na posição de i, pela carta que está na posição que foi sorteada para J.

4. Inserindo Força para as Cartas

Aqui, o procedimento define quem é o manilha, define a força base de cada carta, e dá a força maior para as cartas que são manilha, respeitando a sequência de força das manilhas:

Paus, Copas, Espadas e Ouro.

```
for i := 1 to 40 do
begin
  case baral[i].carta of
    4: baral[i].forca := 1;
    5: baral[i].forca := 2;
    6: baral[i].forca := 3;
    7: baral[i].forca := 4;
    10: baral[i].forca := 5;
    11: baral[i].forca := 6;
    12: baral[i].forca := 7;
    1: baral[i].forca := 8;
    2: baral[i].forca := 9;
    3: baral[i].forca := 10;
  end;

  if (baral[i].carta = manilha) then
  begin
    if baral[i].naipe = 'Ouro'    then baral[i].forca := 11;
    if baral[i].naipe = 'Espadas' then baral[i].forca := 12;
    if baral[i].naipe = 'Copas'   then baral[i].forca := 13;
    if baral[i].naipe = 'Paus'    then baral[i].forca := 14;
  end;
end;
```

Trecho do arquivo linhas 47 a 83.

• (for i:= 1 to 40 do) define a força base das cartas;

• (if (baral[i].carta = manilha) then) define a força da manilha.

5. Cortando o Baralho

Neste procedimento utiliza-se uma lógica para que tanto quanto o computador, quando o jogador seleciona uma posição do baralho (1 a 40) para estar cortando. Essa lógica também respeita a ideia de quem cortou pela última vez, irá cortar na próxima. Esse procedimento de corte do baralho mantém o baralho como pilha, levando a parte que foi cortada para o fim do baralho. Assim, a distribuição ocorre de forma correta.

```
procedure corteBaralho(var baral: iBaralho; var corte: integer; var quemCorta: boolean);
var i, x: integer;
    novoBaralho: iBaralho;
begin
  x := 1;
  if quemCorta = true then //Defini se quem corta é o jogador ou o computador//
  begin
    writeln('          ====================================');
    repeat
      write('          Sua vez de cortar o baralho(1 à 40): ');
      readln(corte);
    until (corte >= 1) and (corte <= 40);
    writeln('          ====================================');
  end
  else
  begin
    corte:= random(39) + 1;
    writeln('          ====================================');
    writeln ('          O computador cortou na posição ', corte, '!');
    writeln('          ====================================');
  end;
```

```
  for i := corte to 40 do
  begin
    novoBaralho[x] := baral[i];
    x := x + 1;
  end;
  for i := 1 to (corte - 1) do
  begin
    novoBaralho[x] := baral[i];
    x := x + 1;
  end;
  baral := novoBaralho;
end;
```

Trecho do arquivo linhas 85 a 118.

• quemCorta é uma variável boolena. Se for igual a true, indica que o jogador deve cortar, se não o computador começa.
• novoBaralho é um array local de 40 posições. Serve para guardar o baralho já embaralhado, onde acaba sendo devolvido a variável baral no fim do procedimento novamente.

6. Entregando as Cartas para os Competidores

É um procedimento pequeno, com a serventia de entregar as cartas aos jogadores após o baralho ser embaralhado, de forma alternada (cada um recebe uma carta por vez).

```
procedure darCartas(var baral: iBaralho; var maoJ, maoC: iBaralho; var jok: iCarta; var manilha: integer; var quemCorta: boolean);
var i, posiJ, posiC, corte: integer;
begin
  posiJ := 1; posiC := 1;
  corteBaralho(baral, corte, quemCorta);
  jok := baral[7]; //A "vira" da manilha//
  inserirForca(baral, jok, manilha);

  for i := 1 to 6 do
  begin
    if (i mod 2 = 1) then  //Condição para alternar quem começa recebendo as cartas//
    begin
      maoJ[posiJ] := baral[i];
      posiJ := posiJ + 1;
    end
    else
    begin
      maoC[posiC] := baral[i];
      posiC := posiC + 1;
    end;
  end;
end;
```

Trecho do arquivo linhas 120 a 141.

• posiC e posiJ definem a posição da mão de cada um para entrega das cartas;
• (if (i mod 2 = 1) then) condição para entregar as cartas de forma alternada;
• (jok := baral[7]) define a carta “vira”, utilizada para definir a manilha no procedimento “inserirForca”.

7. Procedimento de Resposta Caso o Jogo Seja Trucado

Criado com a funcionalidade para o computador responder ao truco, juntamente com a resposta do jogador também. Para quando o jogador trucar, existe uma lógica de probabilidade para a resposta do computador utilizando o Random para “sortear” a escolha do mesmo.

Para o jogador, é mostrado as opções para o jogador, onde deve ser digitado uma escolha do próprio jogador.

Código para computador responder ao truco:

```
begin
  fugiu:= false;
  if pontos_rodada = 1 then
    aux_pontos:= 3
  else
    aux_pontos:= pontos_rodada + 3;
  if ( quemTrucou = 'J' ) then
  begin
     adversario:= 'Computador';
     if pontos_rodada < 9 then
        s:= random(5) + 1
     else
        s2:= random(3) + 1;
     if (s in [1..2]) or (s2 in [1..2]) then
     begin //Computador aceita truco//
        pontos_rodada:= aux_pontos;
        writeln('           Jogo aceito! Valendo ', pontos_rodada);
     end
     else if (s in [3..4]) or (s2 = 3) then
     begin //Computador fugiu do pedido de truco//
        fugiu := true;
        writeln('           O ', adversario, ' correu! Você levou a rodada.');
     end
     else if ( s = 5 ) and ( pontos_rodada < 9 ) then
     begin    //Computador retrucou//
        pontos_rodada:= aux_pontos;
           aux_pontos:= aux_pontos + 3;
           quemTrucou:= 'C';
           quemPede:= 'C';
           truco(pontos_rodada, quemTrucou, quemPede, fugiu)
     end;
  end
```

Lógica para o jogador responder ao truco:

```
 if pontos_rodada = 1 then
   writeln('           --- O COMPUTADOR PEDIU TRUCO! ---')
 else if pontos_rodada = 3 then
   writeln('           --- O COMPUTADOR PEDIU SEIS! ---')
 else if pontos_rodada = 6 then
   writeln('           --- O COMPUTADOR PEDIU NOVE! ---')
 else
   writeln('           --- O COMPUTADOR PEDIU DOZEEE! ---');
 writeln('           --- O que deseja fazer? ---');
 write('           1 - Aceitar   2 - Correr  ');
 if pontos_rodada < 9 then
     writeln(' 3 - Pedir ', aux_pontos + 3);
 repeat
     write('           Escolha: ');
     readln(opcao);
 until (opcao in [1, 2]) or ((opcao = 3) and (pontos_rodada < 9));
 case opcao of
   1: begin //Opção para aceitar//
        pontos_rodada:= aux_pontos;
        writeln('           Jogo aceito! Valendo ', pontos_rodada);
      end;
   2: begin //Opção para correr//
        fugiu := true;
        writeln('           ' ,adversario, ' correu! O Computador leva a rodada.');
      end;
   3: begin //Opção para retrucar//
        pontos_rodada:= aux_pontos;
        aux_pontos:= aux_pontos + 3;
        quemTrucou:= 'J';
        quemPede:= 'J';
        truco(pontos_rodada, quemTrucou, quemPede, fugiu);
      end;
 end;
```

Trecho do arquivo linhas 143 a 217.

• quemTrucou é uma string(“J”= Jogador e “C” = computador) utilizada para verificar quem trucou, e o procedimento rodar o código corretamente para quem precisa responder o pedido de truco;

• “s” e “s2” são variáveis criadas para criar possibilidades maiores de resposta do computador;

• fugiu é uma variável booleana e serve para caso o jogador ou o computador correr do truco;

• aux_pontos é utilizada apenas para auxiliar a variável pontos_rodada. Quando o jogo é trucado, aux_pontos recebe + 3, porém, pontos_rodada só recebe os devidos caso o pedido de truco seja aceitado ou retrucada.

8. Cabeçalho de Informações do Jogo

Um simples procedimento, que serve para mostrar as informações básicas do jogo(Pontuação Geral, Manilha, Placar da Rodada e Valor da Rodada) de forma organizada.

```
procedure mostraCabecalho ( quedasJ, quedasC: integer; manilha, pontos_rodada, pontosC, pontosJ: integer);
begin
  writeln;
  textcolor(lightblue);
    writeln('          ========================================');
    writeln('          PONTUAÇÃO GERAL: Você ', pontosJ, ' x ' ,pontosC, ' Computador');
    writeln('          ========================================');
  textcolor(lightgray);
    writeln('          A MANILHA DA RODADA É O: ', manilha);
    writeln('          Placar da Rodada:   Você ', quedasJ, ' x ', quedasC, ' Computador');
    writeln('          Rodada Valendo: ', pontos_rodada);
    writeln('          ========================================');
  writeln;
end;
```

Trecho do arquivo linhas 283 a 296.

9. Mão de Ferro e Mão de Onze

Para a regra de mão de ferro, existe uma verificação dentro do procedimento de rodada(linha 346), onde verifica se ambos competidores têm 12 pontos, caso contrário, o jogo não demonstra a opção de trucar. Assim, o jogo valerá apenas um ponto, e os competidores jogam de “mano para mano”.

Já em questão da regra de mão de onze, foi criado um procedimento separado para esta regra. Este procedimento imita o procedimento de rodada, trazendo de diferente a opção de aceitar ou recusar a mão de onze para o competidor que estiver com 11 pontos. Caso ele aceitar, o valor da rodada passa a ser 3 pontos, se não, o oponente ganha 1 ponto. Neste procedimento não traz a opção de trucar.

Trecho do arquivo linhas 219 a 281(mão de onze) .

10. Procedimento de Rodada

O procedimento de rodada é como o “coração” do código desenvolvido. Ele trás a lógica de utilização das cartas dos jogadores, os pedidos de truco com suas devidas regras, contabiliza a pontuação dos competidores e organiza quem deve começar a partida(variável quemComeca), respeitando o corte do baralho.

Uma lógica muito interessante neste procedimento é na do computador de pedir truco. Foi

trabalhada a ideia de trazer uma probabilidade maior de o computador pedir o truco caso ele já tenha uma queda da rodada ganha(linhas 434 a 439), sem descartar a chance de ele pedir truco de início.

Foi desenvolvido também ao final do código (linhas 519 a 575) uma lógica para contar as quedas e pontos ganhados de cada competidor. Este trecho de código também considera a regra em casos de empate(se empatar a primeira queda, leva a segunda queda; se empatar a segunda queda, leva quem ganhou a primeira queda; se empatar as duas primeiras quedas, leva quem ganhar a última).

E para finalizar, ao fim de cada queda, e cada rodada, foi trazida a função de limpar a tela (ClrScr), trazendo uma melhor experiência para o usuário, não causando confusão para ler o que o jogo pede e descreve.

Trecho do arquivo linhas 298 a 582.

11. Contando até Doze

O último procedimento do código: procedimento partida. Aqui, apenas é contado se algum dos competidores chegaram a 12 pontos e organiza os procedimentos necessários para que tudo ocorra corretamente durante a partida.

```
procedure partida (var maoJ, maoC, bar: iBaralho; jok: iCarta; manilha: integer; var quemCorta: boolean);
var pontos_gerais, pontosC, pontosJ: integer;
begin
    quemCorta:= true;
    while (pontosC <=11) and (pontosJ <=11) do
    begin
      embaralhar(bar);
      darCartas(bar, maoJ, maoC, jok, manilha, quemCorta);
      rodada ( maoJ, maoC, manilha, pontosC, pontosJ);
      if quemCorta = true then
        quemCorta := false
      else
        quemCorta := true;
      ClrScr;
    end;
    writeln;
    writeln;
    writeln('          ====================');
    if pontosC >= 12 then
      writeln ('          VOCÊ PERDEU À PARTIDA!')
    else
      writeln ('          VOCÊ VENCEU À PARTIDA!');
    writeln('          ====================');
    readkey;
end;
```

Trecho do arquivo linhas 584 a 608.

12. Variáveis Globais e Begin/End Principal

A parte que impressiona. Trazendo apenas seis variáveis globais, e duas linhas de begin/end principal, o jogo acontece perfeitamente! A utilização dos procedimentos se tornou muito integrado(procedimento que chama procedimento), o que acabou deixando pouquíssimas linhas para o Begin/End principal.

```
  var
    baralho: iBaralho;
    maoJogador: iBaralho;
    maoComputador: iBaralho;
    joker: iCarta;
    manilha: integer;
    corteC: boolean;

Begin
  inserirBaralho(baralho);
  partida (maoJogador, maoComputador, baralho, joker, manilha, corteC);
End.
```

Trecho do arquivo linhas 612 a 623.